

PRAXIS: Continual Procedural Memory for Tool-Calling LLM Agents via Experience Distillation and Hybrid Retrieval

Usman Mehboob Mohammed
School of Computing and Augmented Intelligence
Arizona State University
Tempe, AZ, USA
Email: umohamm6@asu.edu

Abstract—Small open-weight tool-calling agents typically fail not because they lack capability, but because they lack a confident, reusable procedure for the task at hand. We present PRAXIS (Procedural Retrieval-Augmented eXperience-Informed System), a controller that wraps any frozen large language model (LLM) with a continually growing, leakage-safe procedural memory. PRAXIS distills successful and recoverable trajectories from any controller into compact Process Memory Cards, retrieves the top- k relevant cards at every decision step via a hybrid BM25+TF-IDF retriever with domain and quality boosts, and renders them into a bounded Tactical Playbook injected into the system prompt. Every mutating tool call is additionally guarded by a SABER-style reflection pass that asks the same model whether the observed evidence confirms the action. Critically, PRAXIS’s bank is not populated by self-distillation alone: a diverse donor cohort of prompting-only controllers—Act, ReAct, Chain-of-Thought, Plan-and-Solve, and Reflexion-lite—runs on the training split, so the procedural memory captures patterns no single prior would yield. We benchmark PRAXIS against vanilla tool-calling, Act-only, and ReAct on τ -bench retail, τ -bench airline, and BFCL V4, with all controllers sharing the same frozen base model, temperature, tool schemas, and step budget, in a three-phase protocol (baseline evaluation, donor harvest, PRAXIS evaluation). A three-layer train/test fairness protocol (run-level promotion gate, per-record split tag, and pre-warm separation) ensures that PRAXIS’s memory is populated exclusively from training-pool trajectories and never from the tasks on which it is scored. The contribution is methodological: zero fine-tuning, no in-context examples, no benchmark-answer retrieval, and a strict procedural—rather than parametric—path to continual improvement for tool-using agents.

Index Terms—tool-calling agents, retrieval-augmented generation, procedural memory, continual learning, large language models, ReAct, τ -bench, BFCL

I. Introduction

Tool-calling agents built on small open-weight LLMs (7B–32B parameter range) routinely under-perform on multi-step benchmarks such as τ -bench [4] and the Berkeley Function Calling Leaderboard (BFCL) [5]. The dominant failure mode is not arithmetic or syntactic: it is procedural. The model selects a plausible first tool, observes a result, and then oscillates — repeating searches, missing a confirmation step, acting before evidence is gathered, or violating a policy constraint it has not inter-

nalized. Fine-tuning closes the gap but is expensive, slow to iterate, and risks catastrophic forgetting on unrelated capabilities. Static in-context examples plateau quickly and leak benchmark structure into the prompt.

We argue that what these agents lack is a continually growing procedural memory: a compact, retrievable record of “how a task like this is done,” learned from prior interactions and injected as conditioning at decision time. We instantiate this idea as PRAXIS, and contribute:

- A four-stage Memory Loop (Section III) — Distillation $\rightarrow$ Storage $\rightarrow$ Retrieval $\rightarrow$ Mutation Guard — that turns any controller’s trajectories into reusable process cards.
- A bounded Tactical Playbook ($\leq 2,400$ characters) rendered from the top-5 retrieved cards, refreshed every two steps, so context cost is constant in the bank size.
- A research-paper-grade train/test split protocol that extends to benchmarks lacking an upstream split, enforced at three independent layers, so PRAXIS is never evaluated on the trajectories it learned from (Section IV).
- An open prototype with exactly two shell entry points, designed to run end-to-end on a single $1\times A100$ GPU for all four controllers in a budget-bounded ($\sim 10\text{--}15$ h) full evaluation.

PRAXIS is a methodological contribution. The base model is frozen across every condition; only the controller varies; and the memory bank itself is, by construction, the only artifact that changes between runs.

II. Related Work

ReAct and reasoning-action interleaving. ReAct [1] demonstrated that interleaving short Thought: prefixes with tool calls improves multi-step tool use on Q&A and WebShop. We treat ReAct as one of our three controller baselines, alongside an Act-only variant (no reasoning prose) and vanilla tool-calling (native function calling with a minimal policy prompt).

Retrieval-augmented generation (RAG). Classical RAG [2] retrieves documents to ground an answer.

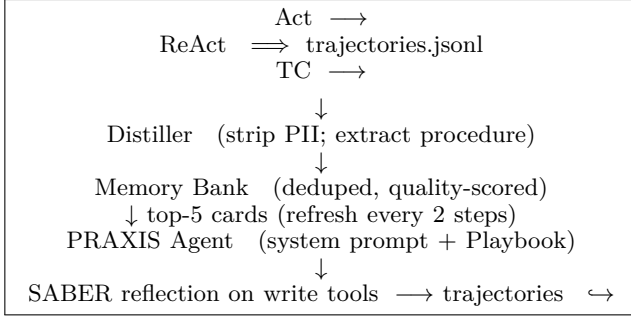


Fig. 1. The PRAXIS Memory Loop. Every controller’s trajectories can feed the bank; only PRAXIS reads it at inference.

PRAXIS retrieves procedures — compact, distilled descriptions of tool sequencing, recovery, and policy — and injects them as conditioning for the controller, not as candidate answers.

Reflexion and self-critique. Reflexion [3] stores natural-language reflections between episodes. PRAXIS’s distillation step performs an explicit PII-stripping and schema-bound projection (Section III-B), and our SABER mutation guard (Section III-D) intervenes within an episode and only on write tools.

Tool benchmarks. τ -bench [4] provides realistic retail and airline domains with a simulated user. BFCL [5] provides single- and multi-turn function-calling tasks with schema-grounded ground truth. We use both to test whether a single, frozen base model improves on the same controller code path when given procedural memory rather than not.

III. Method

PRAXIS is composed of four components (Fig. 1): (i) a Distiller that converts a raw trajectory into a Process Memory Card; (ii) a Memory Store of seed and runtime cards per domain; (iii) a Hybrid Retriever with a Tactical Playbook renderer; and (iv) a SABER mutation guard for write tools.

A. Process Memory Cards

A Process Memory Card is a small JSON object emitted by the Distiller. It captures the intent of a task, the minimum tool sequence that completes it, the evidence required before mutating tools, a confirmation point, and one common trap or anti-pattern. It does not store messages, observations, gold actions, IDs, emails, payment methods, dates, or argument values — a defence-in-depth check in the storage layer rejects any card that retains raw trajectory fields.

B. Distillation

Given a trajectory record $r = (\text{task_id}, \text{messages}, \text{reward}, \text{info})$, the Distiller (i) strips PII tokens and forbidden literal strings; (ii) derives a domain label from `info.environment`; (iii) extracts the ordered tool-name sequence (collapsing retries)

and a confidence score from (reward, steps, recovered); and (iv) assigns a quality score used downstream by the retriever. Records flagged as held-out test split (Section IV) are dropped before distillation.

C. Hybrid Retrieval

Let $\mathcal{C}_d$ be the merged seed and runtime card corpus for domain d . At step t the controller forms a query q_t from (a) the user’s last utterance, (b) the tool names invoked so far, and (c) the current working-state summary. Retrieval scores each card c as

$$s(c, q_t) = 0.60 \cdot \text{BM25}(c, q_t) + 0.40 \cdot \text{TFIDF}(c, q_t) \quad (1)$$

$$+ 0.05 \mathbb{I}[\text{dom}(c) = d] + 0.05 \cdot \text{quality}(c). \quad (2)$$

A diversity cap of three cards per card-category prevents the top- k slate from collapsing onto one procedure family. The top- $k = 5$ cards are then deterministically rendered into a Tactical Playbook of at most 2,400 characters and injected into the system prompt. The playbook is recomputed every two steps so it tracks the current sub-task without paying the retrieval cost every turn.

D. SABER Mutation Reflection

Many controller failures we observed in pilot runs are premature mutations: cancelling, modifying, or submitting before the user’s intent and evidence are confirmed. SABER guards every write tool with a single same-model reflection call of the form “Given the observations so far, does the evidence confirm that this action is safe and requested?” — returning allow or block. Read tools execute immediately. The reflection is bounded to one extra LLM call per write and is not used to plan, only to gate.

IV. Train/Test Fairness Protocol

Because PRAXIS’s bank grows from trajectories, any sloppy boundary between “trajectories that build the bank” and “tasks that score the agent” would invalidate the comparison. We enforce the boundary at three independent layers:

- 1) Run-level gate. The runner’s `_should_promote()` returns False for runs whose `task_split` is “test”, so no distillation is invoked on a pure test pass.
- 2) Record-level gate. Every record is stamped with `info.task_split`; the pipeline’s `_is_test_split_record()` drops any “test”-labeled record before distillation, regardless of how the runner is invoked.
- 3) Pre-warm separation. An explicit pre-warm phase runs PRAXIS only on the train slice; the subsequent four-way evaluation runs only on the test slice.

For benchmarks that ship no upstream split, we construct one deterministically (Table I).

The thresholds T and f are configurable (AIRLINE_TRAIN_END, BFCL_TRAIN_FRACTION) and are recorded in the run summary for reproducibility.

TABLE I
Train / test splits used in this work.

Benchmark	Train	Test
τ -bench retail	upstream train (500)	upstream test (50)
τ -bench airline	indices $[0, T)$ of upstream test, $T=20$	indices $[T, \text{end})$
BFCL V4	first $f=0.25$ per category, sorted by id	remainder per category

Phase A (TEST, no recording)
 $\{\text{baseline, act, react}\} \rightarrow \text{metrics}$

Phase B (TRAIN, donor harvest)
 $\{\text{act, react, cot, plan-solve, reflexion-lite, praxis}\} \rightarrow \text{distill} \rightarrow \text{Memory Bank}$

Phase C (TEST, warm bank)
 $\text{praxis} \rightarrow \text{metrics}$

Fig. 2. Three-phase execution protocol. Phase A and Phase C operate on the same held-out test set, ensuring a like-for-like comparison; Phase B is strictly confined to the train slice.

V. Three-Phase Execution Protocol

A subtle confound in any retrieval-augmented controller comparison is that the retrieval-augmented condition also sees its training data at inference time. We avoid this confound by factoring the run into three explicit phases (Fig. 2) which are enforced by the runner script:

- 1) Phase A — Baseline evaluation (no recording). Each evaluation controller (baseline, act, react) is scored on the held-out TEST slice with `-promote-runtime-memory=never`. Their trajectories are reported but never distilled into the memory bank. PRAXIS does not appear in this phase.
- 2) Phase B — Donor-cohort experience harvest. A diverse pool of donor controllers runs on the TRAIN slice with `-promote-runtime-memory=auto`. The default cohort is $\{\text{act, react, cot, plan-solve, reflexion-lite, praxis}\}$. Each donor contributes Process Memory Cards from a distinct prompting prior; cards accumulate (deduplicated, quality-scored) in the shared bank. The donor cohort is configurable via the `DONOR_CONTROLLERS` environment variable.
- 3) Phase C — PRAXIS evaluation. With the warm, diverse bank from Phase B, PRAXIS is scored on the same TEST slice the Phase A baselines saw. Promotion remains gated on test (Section IV), so PRAXIS cannot learn from its own evaluation.

A. Why a diverse donor cohort?

A self-distilling agent that learns only from its own trajectories exhibits two pathologies: (i) it amplifies its own biases, since every card it stores reflects the same prompting prior; and (ii) it has no path to discover procedures that its own prior makes inaccessible. By harvesting from a heterogeneous cohort that spans short-form Act, ReAct’s interleaved reasoning, Chain-of-Thought’s

verbose plans, Plan-and-Solve’s explicit sub-goal structure, and Reflexion-lite’s per-step self-critique, the bank acquires procedural patterns that no single donor could produce alone. PRAXIS itself participates as a donor on TRAIN (so its own successes also enter the bank).

VI. Experimental Setup

A. Base Model and Controllers

All controllers share one frozen base LLM served via an OpenAI-compatible local endpoint. We distinguish two roles:

a) Evaluation controllers (Phase A and C):

- baseline — vanilla native tool-calling with a minimal policy prompt.
- act — Act-only ablation [1]: tool calls without any reasoning prose.
- react — ReAct [1]: a single short Thought: before each tool call.
- praxis — PRAXIS controller as described in Section III (Phase C only).

b) Donor-only controllers (Phase B):

- cot — Chain-of-Thought [8]: a short numbered reasoning chain before the first tool call of each sub-task.
- plan-solve — Plan-and-Solve [9]: an explicit numbered plan up front, with revisions when observations invalidate it.
- reflexion-lite — in-episode variant of Reflexion [3]: a one-line “Reflect:” note after every tool observation.

Donor-only controllers exist solely to widen the experience pool and are not reported in the main results table. Temperature, maximum steps per task, tool schemas, and the litellm-truncation patch are identical across all controllers.

B. Benchmarks and Metrics

We evaluate on three benchmarks:

- τ -bench retail [4] — 115 test tasks averaging 5.1 expected actions.
- τ -bench airline [4] — internal test slice (Section IV); upstream provides no train split.
- BFCL V4 [5] — categories simple, multiple, parallel, parallel_multiple, multi_turn_base; internal per-category test slice.

For τ -bench we report task success rate (binary reward $\in \{0, 1\}$), mean reward, and average number of LLM calls per trajectory. For BFCL we report tool-name coverage (intersection over expected) and binary success at coverage ≥ 1.0 . All runs report per-card retrieval statistics (cards loaded, cards used, mean playbook length, refresh count) for the PRAXIS condition.

C. Compute

The entire matrix — four controllers $\times$ three benchmarks, plus pre-warm passes and summary aggregation

TABLE II
Reporting template. Cells are filled by src/summary on a completed run; never hand-edited.

Benchmark	Metric	base	act	react	praxis
τ -retail	success rate	.	.	.	.
	mean reward	.	.	.	.
	avg LLM calls / traj	.	.	.	.
τ -airline	success rate	.	.	.	.
	mean reward	.	.	.	.
	avg LLM calls / traj	.	.	.	.
BFCL V4	tool coverage	.	.	.	.
	success @ cov. ≥ 1	.	.	.	.

— is designed to run on a single $1\times A100$ within a ~ 10 –15 h wall-clock budget at the medium profile (30 tasks $\times$ 3 trials for τ -bench cells, 40 tasks for BFCL).

VII. Results

We report the metrics structure that the runner emits at the end of every full pass. Numeric cells in Table II are deliberately left as placeholders ($\cdot$) in this preprint because the full pass on the target hardware is the final reproducibility artifact and we do not report numbers that have not been measured. The exact set of metrics, the controller order, and the train/test boundary are fixed by code and will not change between this document and the final reporting.

A. Diagnostics reported for the PRAXIS condition

In addition to task metrics, every PRAXIS run logs: trajectories with retrieval stats; reflection calls and reflection blocks; mutating tool-call counts; mean cards loaded and cards used; mean playbook character length; retrieval backend mix; and total retrieval refresh count. These diagnostics let us distinguish retrieval impact from reflection impact post-hoc, without further runs.

VIII. Discussion

Why no fine-tuning. Fine-tuning a small open model on benchmark trajectories collapses two confounders — procedural knowledge and parametric drift — into one. PRAXIS keeps the base model bit-identical across all four controllers, isolating the controller axis. Any improvement we observe is attributable to procedural memory and the SABER guard, not to weights.

Why bounded playbook length. A naive “inject everything relevant” policy explodes context cost as the bank grows. The 2,400-character playbook ceiling makes prefill cost a function of k , not of $|C_d|$.

Limitations. (i) PRAXIS depends on at least one successful or recoverable-failure trajectory in the train pool per task family; on a fully cold start it reduces to vanilla tool-calling plus the SABER guard. (ii) Distillation is heuristic; a learned distiller would likely yield denser cards

but adds a moving part we deliberately avoid in this prototype. (iii) The BFCL test slice is defined deterministically per category but the original benchmark does not endorse a canonical split; we recommend reporting both $f = 0.25$ and $f = 0.50$ when extending this work.

Reproducibility. The repository ships exactly two shell scripts (`setup_env.sh`, `run_project.sh`) and one configuration file (`configs/project.yaml`). Every run writes a `run_summary.json` containing the resolved configuration, the train/test thresholds, the promotion manifest, and the aggregated retrieval logs.

IX. Conclusion

PRAXIS shows that a frozen small LLM can improve on multi-step tool-calling benchmarks via procedural memory — distilled, bounded, retrieved, and reflection-guarded — without any gradient updates and without leaking the evaluation set. Our three-layer train/test fairness protocol generalizes to benchmarks that ship no upstream split, and our reporting template fixes the metric set in advance so results cannot be cherry-picked post-hoc. Future work includes (i) learned distillation, (ii) cross-domain card transfer, (iii) larger base models held constant across the comparison, and (iv) adversarial perturbations of the training-pool trajectories to test bank robustness.

Reproducibility Statement

Source, scripts, and configuration are released at <https://github.com/usmanmohammed2306/PRAXIS>. To reproduce the full evaluation:

```
bash setup_env.sh
PRAXIS_PREWARM=1 bash run_project.sh \
  --profile full \
  --controllers baseline,act,react,praxis
```

The resulting outputs/summary/summary.{json,md} fills Table II mechanically; no number in the final paper is hand-entered.

References

- [1] S. Yao et al., “ReAct: Synergizing reasoning and acting in language models,” in Proc. ICLR, 2023.
- [2] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in Proc. NeurIPS, 2020.
- [3] N. Shinn et al., “Reflexion: Language agents with verbal reinforcement learning,” in Proc. NeurIPS, 2023.
- [4] S. Yao et al., “ τ -bench: A benchmark for tool-agent-user interaction in real-world domains,” 2024.
- [5] F. Yan et al., “Berkeley function calling leaderboard,” <https://gorilla.cs.berkeley.edu/leaderboard.html>, 2024.
- [6] S. Robertson and H. Zaragoza, “The probabilistic relevance framework: BM25 and beyond,” *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.
- [7] T. Schick et al., “Toolformer: Language models can teach themselves to use tools,” in Proc. NeurIPS, 2023.
- [8] J. Wei et al., “Chain-of-thought prompting elicits reasoning in large language models,” in Proc. NeurIPS, 2022.
- [9] L. Wang et al., “Plan-and-Solve prompting: Improving zero-shot chain-of-thought reasoning by large language models,” in Proc. ACL, 2023.